ⓘ Event ends in 2 days 2 hours 54 minutes. ✕

Component architecture transforms your design system into a scalable, maintainable React application. This section creates detailed specifications for React components, TypeScript integration, state management, and reusable component libraries.

Exercise 2.1: React Component Hierarchy

Create comprehensive React component architecture specifications:

~~~
Design detailed React component hierarchy for our chatbot application:

**Use the NPM Registry MCP server to research current React patterns and libraries, then create:**

**Application Component Structure**:
```typescript
interface AppComponentHierarchy {
  App: {
    purpose: "Root application component with providers";
    children: ["Router", "ThemeProvider", "AuthProvider", "ErrorBoundary"];
    responsibilities: ["Global state", "Route configuration", "Error handling"];
  };

  Layout: {
    purpose: "Main application layout wrapper";
    children: ["Header", "Sidebar", "MainContent", "Footer"];
    variants: ["authenticated", "unauthenticated", "mobile"];
  };

  ChatInterface: {
    purpose: "Core chat functionality container";
    children: ["MessageList", "MessageInput", "TypingIndicator"];
    state: ["messages", "isLoading", "error", "conversationId"];
  };
}
```

**Component Props and TypeScript Interfaces**:
```typescript
// Message Component Interface
interface MessageProps {
  id: string;
  content: string;
  role: 'user' | 'assistant' | 'system';
  timestamp: Date;
  status?: 'sending' | 'sent' | 'error';
  onRetry?: () => void;
  onEdit?: (newContent: string) => void;
  onDelete?: () => void;
}

// Chat Input Component Interface
interface ChatInputProps {
  onSendMessage: (message: string) => Promise<void>;
  disabled?: boolean;
  placeholder?: string;
  maxLength?: number;
  isLoading?: boolean;
  error?: string;
}

// Conversation List Component Interface
interface ConversationListProps {
  conversations: Conversation[];
  activeConversationId?: string;
  onSelectConversation: (id: string) => void;
  onCreateConversation: () => void;
  onDeleteConversation: (id: string) => void;
  isLoading?: boolean;
}
```
~~~

Expected Outcome

- Comprehensive React component hierarchy with clear responsibilities
- TypeScript interfaces for all component props and state
- Component composition patterns for reusability and maintainability
- Clear separation of concerns between presentation and business logic
- Scalable architecture supporting future feature additions

Exercise 2.2: State Management Architecture

Design comprehensive state management using modern React patterns:

~~~
Create detailed state management specifications for our chatbot application:

**Global State Architecture**:
```typescript
// Application State Structure
interface AppState {
  auth: {
    user: User | null;
    isAuthenticated: boolean;
    isLoading: boolean;
    error: string | null;
  };

  chat: {
    conversations: Record<string, Conversation>;
    activeConversationId: string | null;
    messages: Record<string, Message[]>;
    isTyping: boolean;
    error: string | null;
  };

  ui: {
    sidebarOpen: boolean;
    theme: 'light' | 'dark' | 'system';
    notifications: Notification[];
    isOnline: boolean;
  };
}

// Context Providers Structure
interface ContextProviders {
  AuthContext: {
    state: "Authentication state and user information";
    actions: ["login", "logout", "refreshToken", "updateProfile"];
  };

  ChatContext: {
    state: "Chat conversations and messages";
    actions: ["sendMessage", "loadConversation", "deleteConversation"];
  };

  UIContext: {
    state: "UI state and user preferences";
    actions: ["toggleSidebar", "setTheme", "addNotification"];
  };
}
```

**Custom Hooks for State Management**:
```typescript
// Authentication Hook
interface UseAuthReturn {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (credentials: LoginCredentials) => Promise<void>;
  logout: () => void;
  refreshToken: () => Promise<void>;
  error: string | null;
}

// Chat Hook
interface UseChatReturn {
  conversations: Conversation[];
  activeConversation: Conversation | null;
  messages: Message[];
  sendMessage: (content: string) => Promise<void>;
  loadConversation: (id: string) => Promise<void>;
  createConversation: () => Promise<string>;
  isLoading: boolean;
  error: string | null;
}

// API Integration Hook
interface UseApiReturn<T> {
  data: T | null;
  isLoading: boolean;
  error: string | null;
  refetch: () => Promise<void>;
  mutate: (data: Partial<T>) => Promise<void>;
}
```

Research current React state management patterns and libraries using the NPM Registry MCP server.
~~~

Expected Outcome

- Comprehensive state management architecture with clear data flow
- Custom hooks for encapsulating business logic and API interactions
- Context providers for global state management
- TypeScript interfaces ensuring type safety throughout the application
- Performance-optimized state updates and re-rendering patterns

Exercise 2.3: Component Library and Reusability

Create a comprehensive component library with reusable patterns:

~~~
Design detailed component library specifications for maximum reusability:

**Base Component Library**:
```typescript
// Button Component with Variants
interface ButtonProps {
  variant: 'primary' | 'secondary' | 'outline' | 'ghost' | 'danger';
  size: 'sm' | 'md' | 'lg';
  disabled?: boolean;
  loading?: boolean;
  icon?: ReactNode;
  iconPosition?: 'left' | 'right';
  fullWidth?: boolean;
  onClick?: () => void;
  children: ReactNode;
}

// Input Component with Validation
interface InputProps {
  type: 'text' | 'email' | 'password' | 'textarea';
  label: string;
  placeholder?: string;
  value: string;
  onChange: (value: string) => void;
  error?: string;
  disabled?: boolean;
  required?: boolean;
  maxLength?: number;
  rows?: number; // for textarea
}

// Modal Component with Accessibility
interface ModalProps {
  isOpen: boolean;
  onClose: () => void;
  title: string;
  size?: 'sm' | 'md' | 'lg' | 'xl';
  closeOnOverlayClick?: boolean;
  closeOnEscape?: boolean;
  children: ReactNode;
}
```

**Compound Component Patterns**:
```typescript
// Card Component with Composition
interface CardComponent {
  Card: React.FC<CardProps>;
  Header: React.FC<CardHeaderProps>;
  Body: React.FC<CardBodyProps>;
  Footer: React.FC<CardFooterProps>;
}

// Usage Example:
// <Card>
//   <Card.Header>Title</Card.Header>
//   <Card.Body>Content</Card.Body>
//   <Card.Footer>Actions</Card.Footer>
// </Card>

// Dropdown Component with Composition
interface DropdownComponent {
  Dropdown: React.FC<DropdownProps>;
  Trigger: React.FC<DropdownTriggerProps>;
  Content: React.FC<DropdownContentProps>;
  Item: React.FC<DropdownItemProps>;
  Separator: React.FC<{}>;
}
```

**Higher-Order Components and Render Props**:
```typescript
// WithAuth HOC for Protected Components
interface WithAuthProps {
  requireAuth?: boolean;
  redirectTo?: string;
  fallback?: ReactNode;
}

// WithLoading HOC for Async Components
interface WithLoadingProps {
  isLoading: boolean;
  fallback?: ReactNode;
  error?: string | null;
}

// Render Props Pattern for Data Fetching
interface DataFetcherProps<T> {
  url: string;
  children: (data: {
    data: T | null;
    loading: boolean;
    error: string | null;
    refetch: () => void;
  }) => ReactNode;
}
```

Research current React component patterns and composition techniques using the NPM Registry MCP server.
~~~

Expected Outcome

- Comprehensive component library with consistent API patterns
- Reusable components with proper TypeScript interfaces
- Compound component patterns for flexible composition
- Higher-order components and render props for cross-cutting concerns
- Component documentation and usage examples

Exercise 2.4: Performance Optimization Patterns

Implement performance optimization strategies for React components:

~~~
Create detailed performance optimization specifications:

**Memoization and Optimization Strategies**:
```typescript
// Memoized Components
interface OptimizedComponentPatterns {
  // React.memo for preventing unnecessary re-renders
  MemoizedMessage: React.MemoizedComponent<MessageProps>;

  // useMemo for expensive calculations
  expensiveCalculation: "useMemo(() => heavyComputation(data), [data])";

  // useCallback for stable function references
  stableCallback: "useCallback((id: string) => handleClick(id), [handleClick])";

  // Custom hooks with memoization
  optimizedHook: "Custom hooks that memoize return values";
}

// Virtual Scrolling for Large Lists
interface VirtualScrollProps {
  items: any[];
  itemHeight: number;
  containerHeight: number;
  renderItem: (item: any, index: number) => ReactNode;
  overscan?: number;
}

// Lazy Loading Components
interface LazyLoadingPatterns {
  // React.lazy for code splitting
  LazyComponent: "React.lazy(() => import('./Component'))";

  // Intersection Observer for image lazy loading
  LazyImage: "Load images when they enter viewport";

  // Progressive loading for chat history
  InfiniteScroll: "Load more messages as user scrolls";
}
```

**Bundle Optimization Strategies**:
```typescript
// Code Splitting Patterns
interface CodeSplittingStrategy {
  routeBasedSplitting: {
    pattern: "Split by routes using React.lazy";
    implementation: "Separate bundles for each major route";
    fallback: "Loading components for each split";
  };

  featureBasedSplitting: {
    pattern: "Split by features or components";
    implementation: "Lazy load heavy components";
    criteria: "Components > 50KB or rarely used";
  };

  vendorSplitting: {
    pattern: "Separate vendor libraries";
    implementation: "Split React, utilities, and app code";
    caching: "Vendor bundles cached longer";
  };
}

// Tree Shaking Optimization
interface TreeShakingPatterns {
  importStrategy: "Import only needed functions from libraries";
  bundlerConfig: "Configure webpack/vite for optimal tree shaking";
  librarySelection: "Choose libraries that support tree shaking";
  sideEffectsFree: "Mark packages as side-effects free";
}
```

**Runtime Performance Monitoring**:
```typescript
// Performance Monitoring
interface PerformanceMonitoring {
  // React DevTools Profiler integration
  profilerIntegration: {
    onRender: "(id, phase, actualDuration) => logPerformance(...)";
    components: "Wrap expensive components with Profiler";
    analysis: "Identify performance bottlenecks";
  };

  // Web Vitals monitoring
  webVitals: {
    FCP: "First Contentful Paint < 1.8s";
    LCP: "Largest Contentful Paint < 2.5s";
    FID: "First Input Delay < 100ms";
    CLS: "Cumulative Layout Shift < 0.1";
  };

  // Custom performance metrics
  customMetrics: {
    messageRenderTime: "Time to render new message";
    chatLoadTime: "Time to load conversation";
    inputResponseTime: "Input lag measurement";
  };
}
```

Research current React performance optimization techniques using the NPM Registry MCP server.
~~~

Expected Outcome

- Comprehensive performance optimization strategies for React components
- Memoization patterns preventing unnecessary re-renders
- Code splitting and bundle optimization for faster loading
- Virtual scrolling and lazy loading for large datasets
- Performance monitoring and measurement tools

Exercise 2.5: Testing Architecture for Components

Create comprehensive testing specifications for React components:

~~~
Design detailed testing architecture for our React components:

**Testing Strategy and Tools**:
```typescript
// Testing Framework Configuration
interface TestingFramework {
  unitTesting: {
    framework: "Jest with React Testing Library";
    coverage: "Minimum 80% code coverage";
    patterns: "Test behavior, not implementation";
  };

  integrationTesting: {
    framework: "React Testing Library with MSW";
    scope: "Component integration with APIs";
    patterns: "User-centric testing approach";
  };

  e2eTesting: {
    framework: "Playwright for cross-browser testing";
    scope: "Critical user journeys";
    patterns: "Page Object Model";
  };
}

// Component Testing Patterns
interface ComponentTestPatterns {
  // Render testing
  renderTests: {
    pattern: "Test component renders without crashing";
    props: "Test with various prop combinations";
    states: "Test different component states";
  };

  // Interaction testing
  interactionTests: {
    pattern: "Test user interactions (click, type, etc.)";
    events: "Test event handlers are called correctly";
    state: "Test state changes from interactions";
  };

  // Accessibility testing
  a11yTests: {
    pattern: "Test keyboard navigation and screen readers";
    tools: "jest-axe for automated accessibility testing";
    manual: "Manual testing with assistive technology";
  };
}
```

**Mock Strategies and Test Utilities**:
```typescript
// API Mocking with MSW
interface MockingStrategy {
  apiMocking: {
    tool: "Mock Service Worker (MSW)";
    patterns: "Mock API responses for testing";
    scenarios: "Success, error, and loading states";
  };

  componentMocking: {
    pattern: "Mock heavy or external components";
    implementation: "Jest manual mocks";
    criteria: "Mock components that don't add test value";
  };

  hookMocking: {
    pattern: "Mock custom hooks for isolated testing";
    implementation: "Jest mock functions";
    scope: "Test component logic separately from hooks";
  };
}

// Test Utilities and Helpers
interface TestUtilities {
  renderHelpers: {
    renderWithProviders: "Render components with all providers";
    renderWithRouter: "Render components with routing context";
    renderWithAuth: "Render components with authentication";
  };

  userEventHelpers: {
    typeMessage: "Helper for typing in chat input";
    clickButton: "Helper for button interactions";
    navigateChat: "Helper for chat navigation";
  };

  assertionHelpers: {
    expectMessage: "Assert message appears in chat";
    expectLoading: "Assert loading states";
    expectError: "Assert error states and messages";
  };
}
```

**Continuous Integration Testing**:
```typescript
// CI/CD Testing Pipeline
interface CITestingPipeline {
  preCommitHooks: {
    linting: "ESLint and Prettier checks";
    typeChecking: "TypeScript compilation";
    unitTests: "Fast unit test execution";
  };

  pullRequestChecks: {
    fullTestSuite: "All tests including integration";
    coverageReport: "Code coverage reporting";
    visualRegression: "Chromatic visual testing";
  };

  deploymentTesting: {
    e2eTests: "Full end-to-end test suite";
    performanceTests: "Lighthouse CI performance testing";
    accessibilityTests: "Automated accessibility scanning";
  };
}
```

Set up Agent Hooks to automatically run tests when components are modified and update test documentation.

Research current React testing best practices and tools using the NPM Registry MCP server.
~~~

Expected Outcome

- Comprehensive testing strategy covering unit, integration, and e2e testing
- Component testing patterns focusing on user behavior
- Mock strategies for APIs, components, and hooks
- Test utilities and helpers for consistent testing patterns
- CI/CD integration with automated testing and quality gates

Key Deliverables

After completing the Component Architecture exercises, you should have:

1. **React Component Hierarchy**: Scalable component structure with TypeScript interfaces
2. **State Management Architecture**: Modern React patterns with custom hooks and context
3. **Component Library**: Reusable components with consistent API patterns
4. **Performance Optimization**: Memoization, code splitting, and monitoring strategies
5. **Testing Architecture**: Comprehensive testing framework with automation

Integration with Advanced Features

Your component architecture leverages:

- **NPM Registry MCP Server**: For researching React libraries and patterns
- **Agent Steering**: For applying React and TypeScript best practices
- **Agent Hooks**: For automated testing and code quality validation

Next Steps

With your component architecture established, you're ready to implement performance optimization strategies that ensure your chatbot application loads quickly and runs smoothly. The next section will focus on bundle optimization, caching strategies, Progressive Web App features, and Core Web Vitals compliance.

Previous Next